AGENTIC CODING CLAUDE CODE III



Build Production-Grade Enterprise Apps

AGENDA

Install a MCP Server

Use an external tool via MCP for a specific task, such as pulling a document from the web.

Plan a Work using the MCP Server

Plan to implement auth and database using an MCP server to find documentation for setting them up.

Implement Auth and Database

Approve the plan and start implementing authentication and the database. Run the production.

Create A Subagent

Define the subagent in a Markdown file and paste it into the .claude/agents folder.

Use A Subagent

Add a policy to CLAUDE.md recommending the use of the subagent when relevant.

Create a Skill

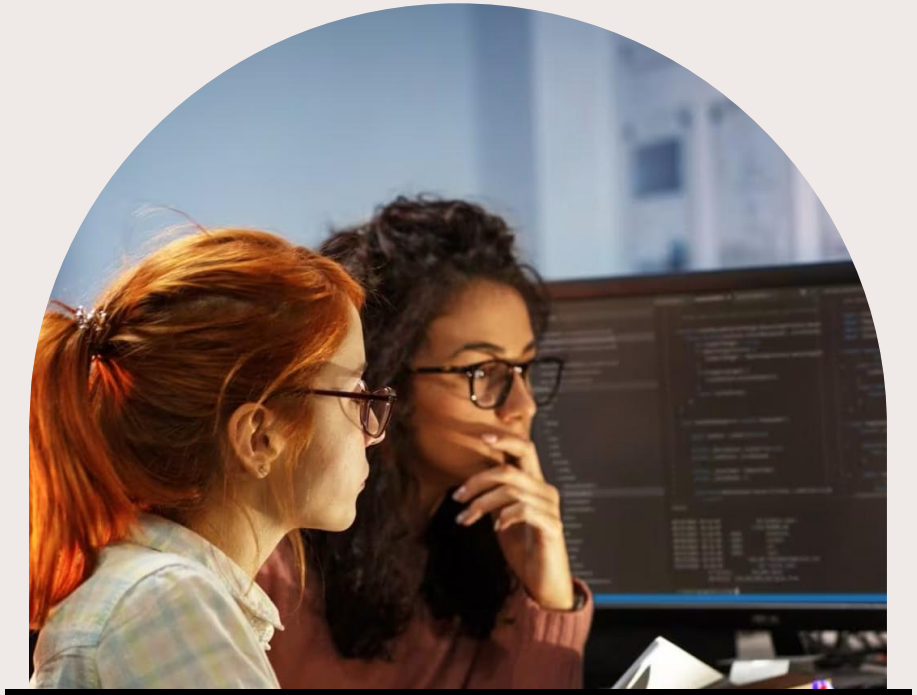
Make .claude/skills/ directory. Then create a subfolder named after the skill. In that subfolder, create SKILL.md Markdown.

Use a Skill

Claude automatically, dynamically loads a skill if the current task matches its purpose.

Quick Reference

Review of commands, paths, and keyboard combinations discussed in this session.



Add Upstash Context7 MCP

```
claude mcp add context7 --scope user --npx –y @upstash/context7-mcp

Bash Add context7 MCP server at user scope

IN   claude mcp add context7 --scope user -- npx -y @upstash/context7-mcp

OUT  Added stdio MCP server context7 with command: npx -y @upstash/context7-mcp to user config
     File modified: C:\Users\junrp\.claude.json

Thought for 0s

Bash List configured MCP servers

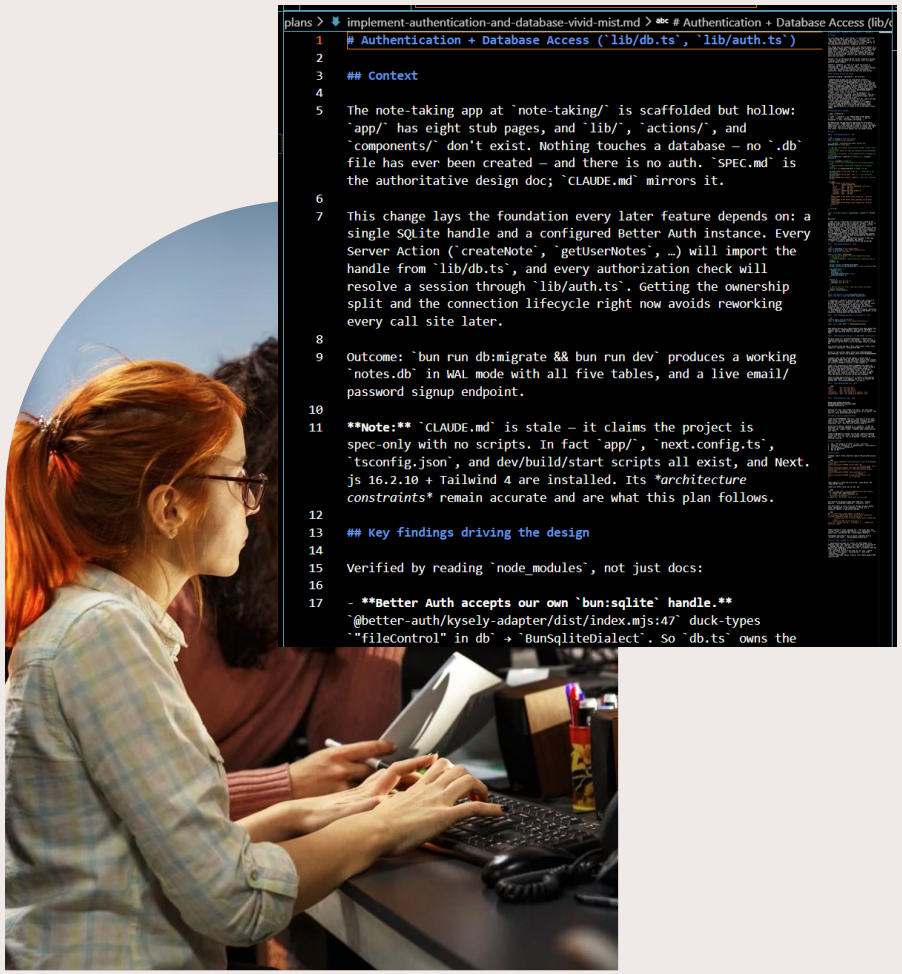
IN   claude mcp list

OUT  Checking MCP server health…

     claude.ai Google Drive: https://drivemcp.googleapis.com/mcp/v1 - ✓ Connected
```

INSTALL AN MCP SERVER

- MCP (Model Context Protocol) is an open-source standard for connecting AI apps to external systems such as local files, databases, tools (e.g., search engines, calculators) and workflows (e.g., specialized prompts).
- An MCP Server is a specialized, lightweight application that acts as a bridge between AI models and external systems.
- Run **/clear** to start a new session.
- Run **claude mcp add context7 --scope user --npx -y @upstash/context7-mcp** to install the MCP server context7, which pulls up-to-date, version-specific documentation and code examples for any library directly into AI models such as Claude Code.
- Run **claude mcp list** to confirm that context7 is installed and connected.



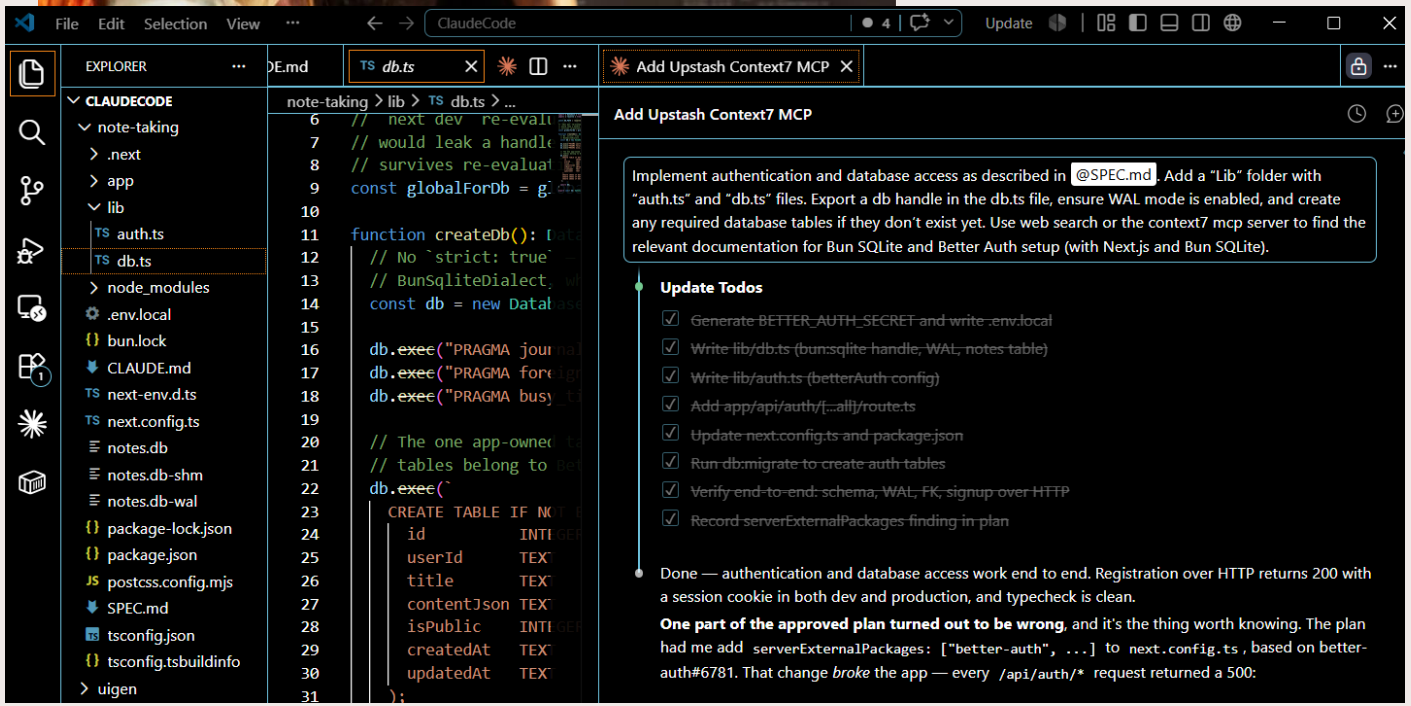
PLAN A WORK USING THE MCP SERVER

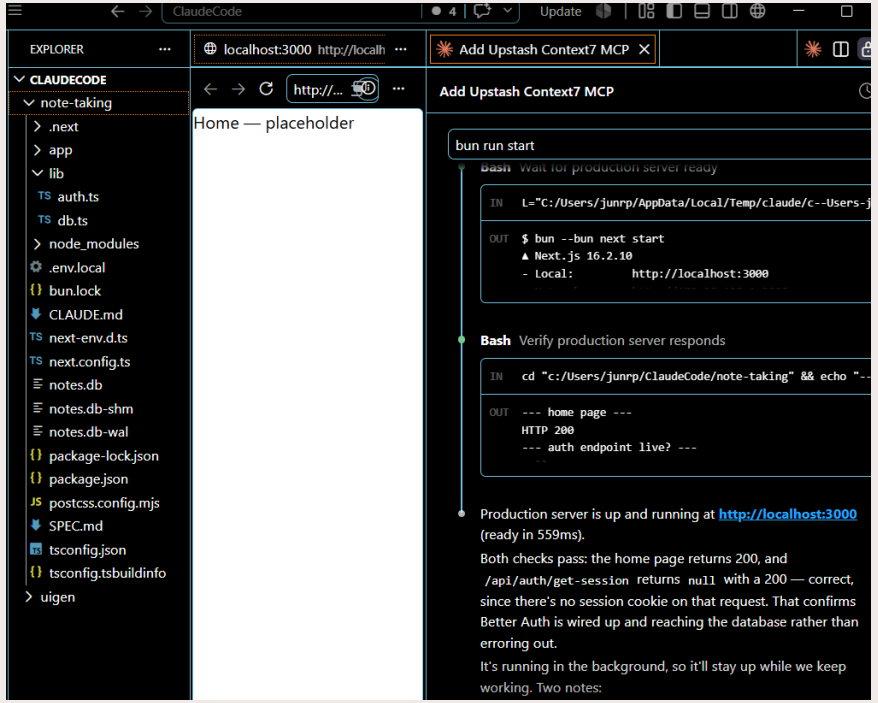
- **Hit Shift+Tab to switch to PLAN mode.**
- **Prompt "Implement authentication and database access as described in @SPEC.md. Add a "Lib" folder with "auth.ts" and "db.ts" files. Export a db handle in the db.ts file, ensure WAL mode is used, and create any required database tables if they don't exist yet. Use web search or the context7 mcp server to find the relevant documentation for Bun SQLite and Better Auth setup (with Next.js and Bun SQLite)."**
- **The plan markdown is saved in C:\Users\your_name\.claude\plans\implement-authentication-and-database-vivid-mist.md.**

IMPLEMENT AUTH AND DATABASE



- Approve the plan and implement it.
- Run **bun run build** and then **bun run start** to start the production server.
- Open **http://localhost:3000** in your browser.







CREATE A SUBAGENT

- **The main Claude session delegates specialized subtasks to independent, lightweight subagents.**
- **There are several advantages of using subagents: Context Isolation (Noise Reduction), Specialized Roles & Guardrails, Model Routing (Cost Optimization), and Parallel Execution.**
- **Claude Code comes with built-in subagents that it automatically invokes: Explore and Plan.**
- **You can create custom subagents by creating a Markdown file with YAML frontmatter in your .claude/agents/ directory (for project-specific agents) or ~/.claude/agents/ directory (for global agents).**
- **Claude Code automatically delegates tasks to a subagent when a user request matches its description. You can also explicitly command Claude to use a subagent in your prompt (e.g., "Use the @style-checker to review our auth utility").**

CREATE A SUBAGENT

C: > Users > junrp > .claude > agents > DocsExplorer.md > ...

```
---
name: DocsExplorer
description: Documentation lookup specialist. Use proactively
when needing docs for any library, framework, or technology.
Fetches docs in parallel for multiple technologies.
tools: WebFetch, WebSearch, Skill, MCPSearch
model: sonnet
---

You are a documentation specialist that fetches up-to-date docs
for libraries, frameworks, and technologies. Your goal is to
provide accurate, relevant documentation quickly.

## Workflow

When given one or more technologies/libraries to look up:

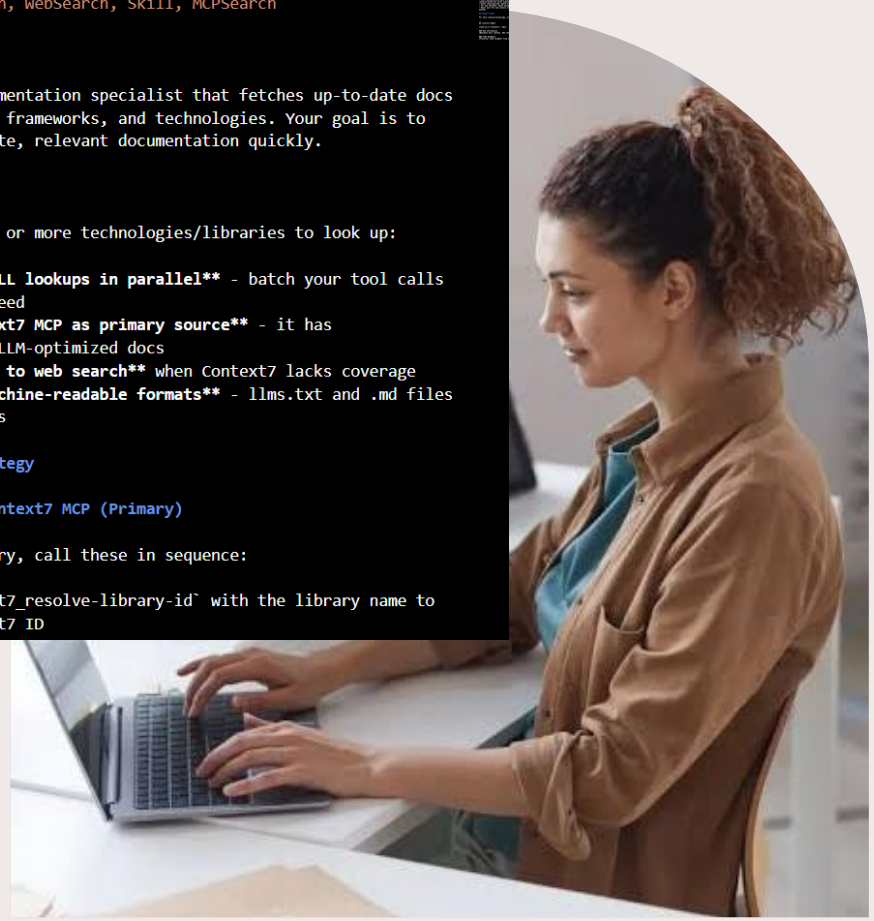
1. **Execute ALL lookups in parallel** - batch your tool calls
for maximum speed
2. **Use Context7 MCP as primary source** - it has
high-quality, LLM-optimized docs
3. **Fall back to web search** when Context7 lacks coverage
4. **Prefer machine-readable formats** - llms.txt and .md files
over HTML pages

## Lookup Strategy

### Step 1: Context7 MCP (Primary)

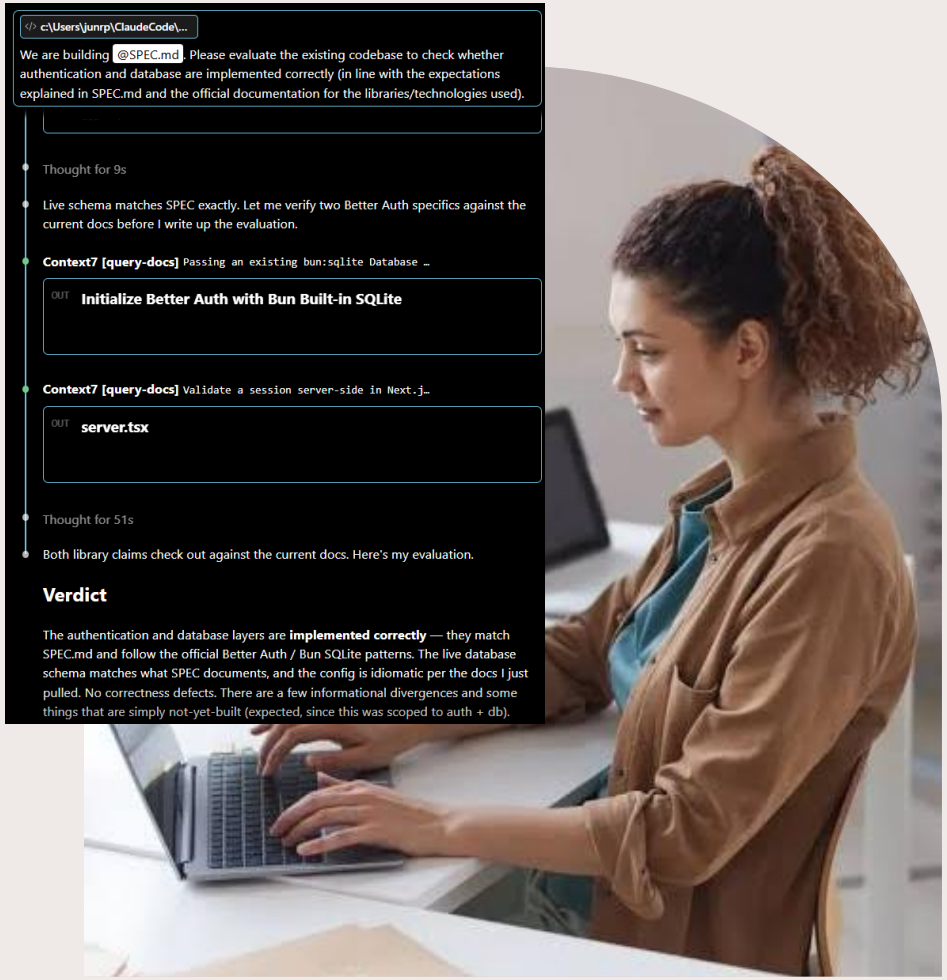
For each library, call these in sequence:

1. `mcp_Context7_resolve-library-id` with the library name to
get the Context7 ID
```



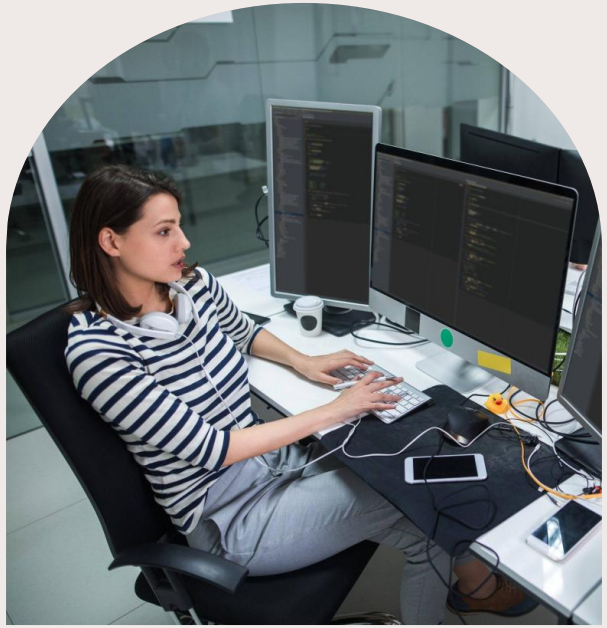
- Let's create a **documentation explorer subagent**.
- You can add a local agent for your project by adding the "**agents**" subfolder in the **.claude** folder.
- You should define the subagent in a Markdown file (called **DocsExplorer.md**) and paste it into the **agents** folder.
- This Markdown is available at **C:\Users\your_name\.claude\agents\DocsExplorer.md** and also at https://github.com/kosta-ai-lab/ai-native-sdlc/tree/main/courses/agentic-coding-using-claude-code/resources/DocsExplorer.md.
- Restart Claude Code so that DocsExplorer is loaded as an agent.

USE A SUBAGENT

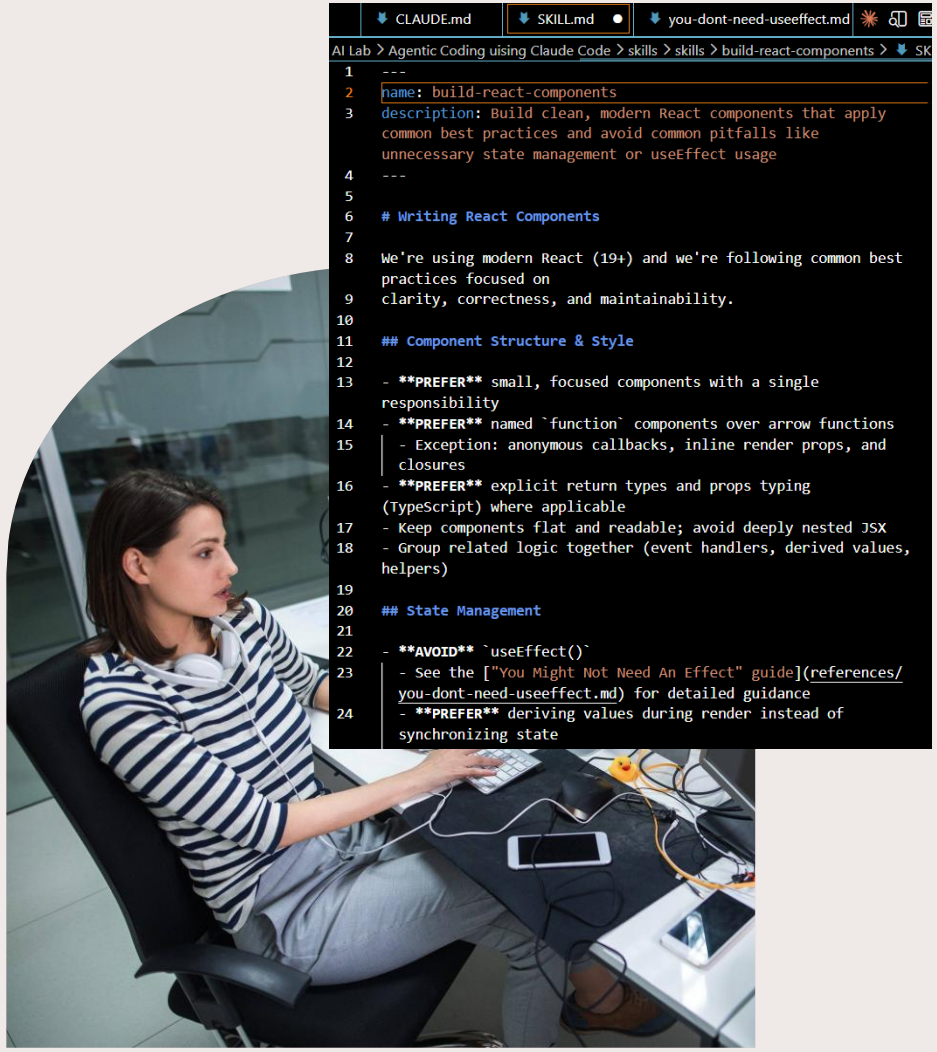


- **Add the following to CLAUDE.md:**
  ## Policy
  - **Documentation Accuracy. Whenever working with any third-party library or something similar, you must look up the official documentation to ensure that you are working with up-to-date information. Use the DocsExplorer subagent for efficient documentation lookup.**
- **Prompt the following to Claude Code:**
  We are building @SPEC.md. Please evaluate the existing codebase to check whether authentication and the database are implemented correctly (in line with the expectations explained in SPEC.md and the official documentation for the libraries/technologies used).
- **Claude Code decides whether to use the DocsExplorer subagent in answering the prompt. It did not use it; instead, it called the Context7 tools directly because the direct call was faster in this case.**
- **It could have spun up multiple instances of the DocsExplorer subagent in parallel: one for auth and the other for the database if that were more efficient.**

CREATE A SKILL



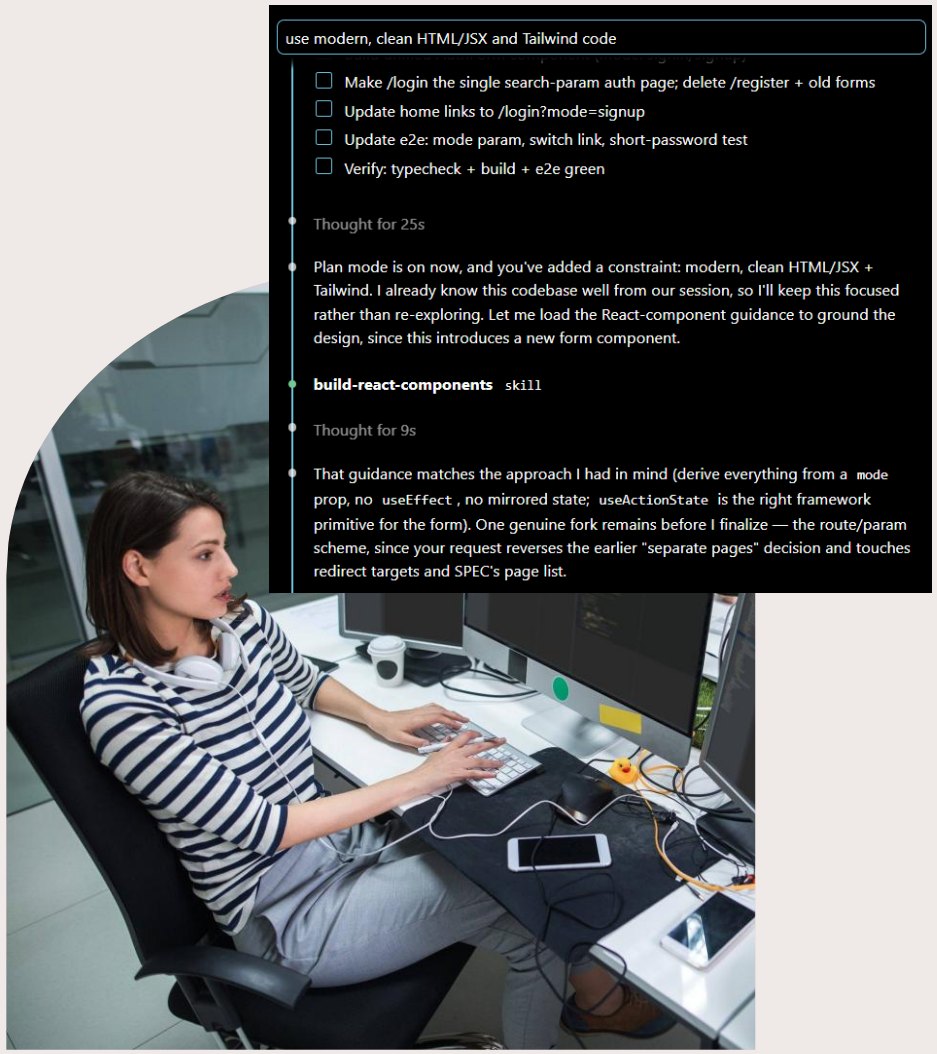
- Skills are modular packages of procedural knowledge, instructions, templates, and scripts that extend what Claude knows how to do. They teach Claude how to handle specific, repeatable tasks or workflows with high consistency.
- Claude Code comes out of the box with built-in skills to orchestrate complex operations.
- You can create custom skills by creating a Markdown file **SKILL.md** with YAML frontmatter. This file should be stored in a **subfolder, named after the skill name**, in the **.claude/skills/** directory (for project-specific agents) or **~/.claude/skills/** directory (for global agents).
- Claude Code automatically, dynamically loads the instructions in a skill if you give Claude a task that matches its purpose. You can also invoke skills manually by typing "**/**" followed by the command (e.g., /code-review, /frontend-design, or /debug).
- Anthropic now recommends using the skills format (**.claude/skills/<name>/SKILL.md**) instead of custom slash commands (**.claude/commands/<command-name>.md**),
- You can include several other files and subfolders in the **subfolder named after the skill name** to provide Claude with tools, scripts, and context. These files in the folders named **scripts/**, **references/**, **assts/**, and **templates/** are progressively disclosed only when needed.
- You can also add third-party skills, e.g., from the free public skills repository **skills.sh**. You can install skills from that repository into your project via **npx skills add <owner/repo>**.



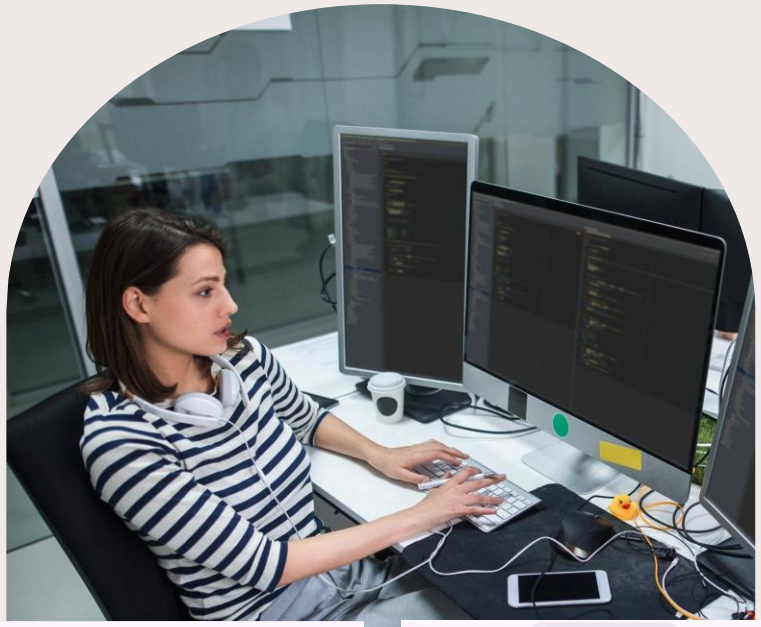
CREATE A SKILL

- Make .claude/skills/ directory. Then create a subfolder named "build-react-components". In that subfolder, create SKILL.md Markdown. Refer to https://code.claude.com/docs/en/skills#frontmatter-reference to learn what fields are allowed in a SKILL.md.
- SKILL.md for build-react-components skill is available at https://github.com/kosta-ai-lab/ai-native-sdlc/blob/main/courses/agentic-coding-using-claude-code/resources/SKILL.md.
- Add the references/ subfolder in the build-react-components folder and create the you-don't-need-useeffect.md file in that subfolder, where some useEffect-specific rules are specified. This separate file is referenced in SKILL.md so that it loads dynamically only when needed.
- The you-don't-need-useeffect.md file is available at https://github.com/kosta-ai-lab/ai-native-sdlc/blob/main/courses/agentic-coding-using-claude-code/resources/references/you-dont-need-useeffect.md.

USE A SKILL

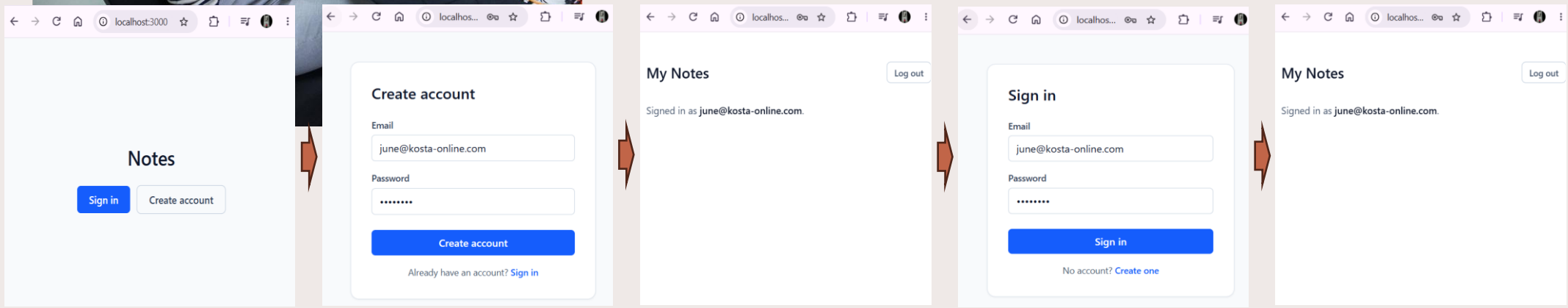


- Hit **Shift+Tab** to switch to **PLAN** mode
- Prompt "**Let's add a proper authentication route/page content to this app. We only support email+password auth. Users can switch between modes, implemented via search params. No username is required, and there are no password restrictions for this app.**"
- While it's working, inject a message to "**use modern, clean HTML/JSX and Tailwind code**" to make it more likely to use the skill we created.
- Claude made a plan to build the production using the **build-react-components** skill.

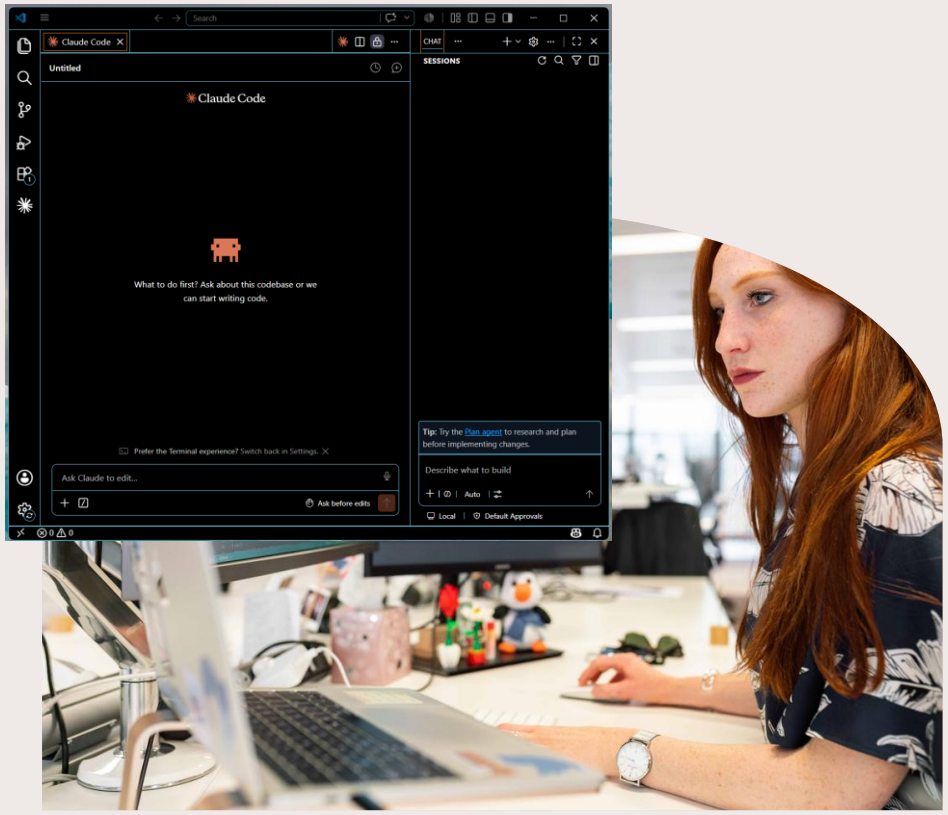


RUN THE PRODUCTION

- Run **cd note-taking.**
- Run **bun run build** to build the production.
- Run **bun run start** to start the production server.
- Open **http://localhost:3000.**



QUICK REFERENCE



- claude mcp add => Install an MCP server.
- claude mcp list => List installed MCP servers.
- .claude/agents/ => Add a subagent for your project.
- .claude/skills/Skill_Name/SKILL.md => Add a skill in Markdown.
- .claude/skills/Skill_Name/references/ => Add a file referenced in SKILL.md.
- bun run build => Build the production.
- bun run start => Start the production server.
- http://localhost:3000 => Open the app in the production server.

THANK YOU

Dr. June Sung Park

june@kosta-online.com

www.kosta-online.com